Lab 4: Gaussian Processes

Manfred Clase (mancl774)

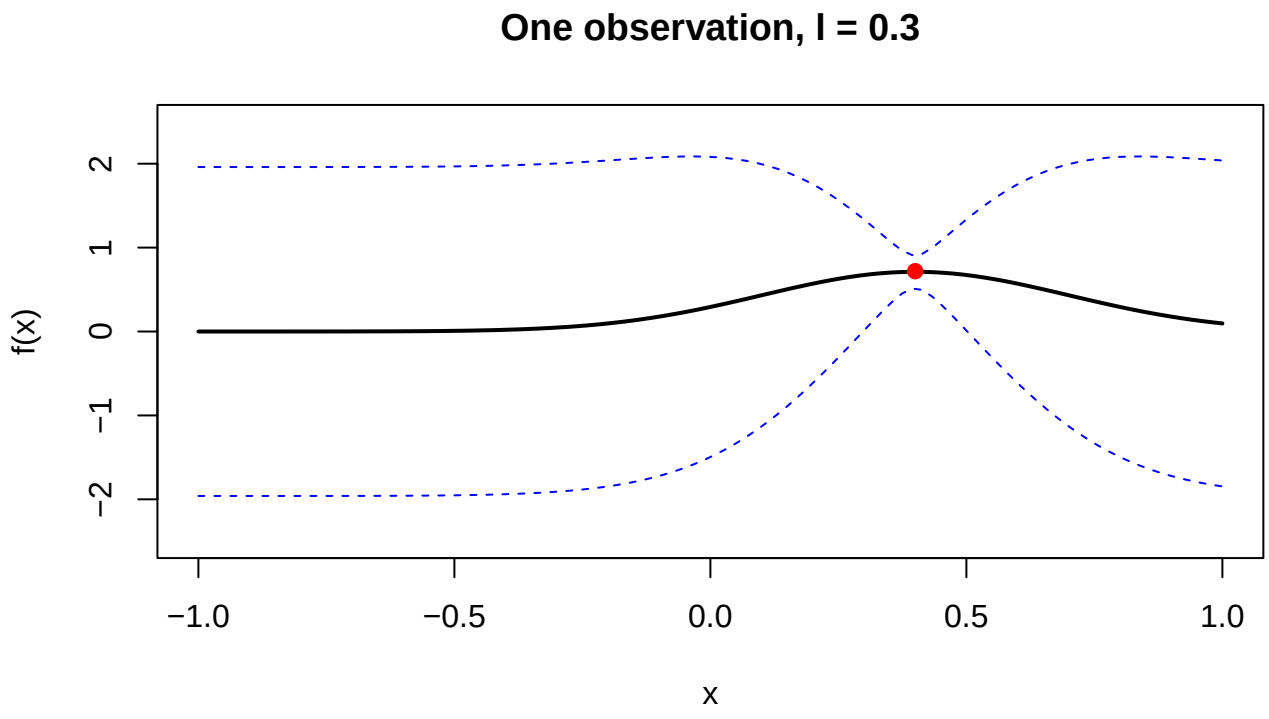
2026-10-11

2.1 Implementing GP Regression

The function `posteriorGP` implements Algorithm 2.1 in Rasmussen and Williams. It uses the Cholesky decomposition of $K + \sigma_n^2 I$ and returns the posterior mean and variance of f at the points in `XStar`. The kernel is passed in as a separate function, so the hyperparameters σ_f and ℓ are set in the kernel and not in `posteriorGP`. The code is listed in the appendix.

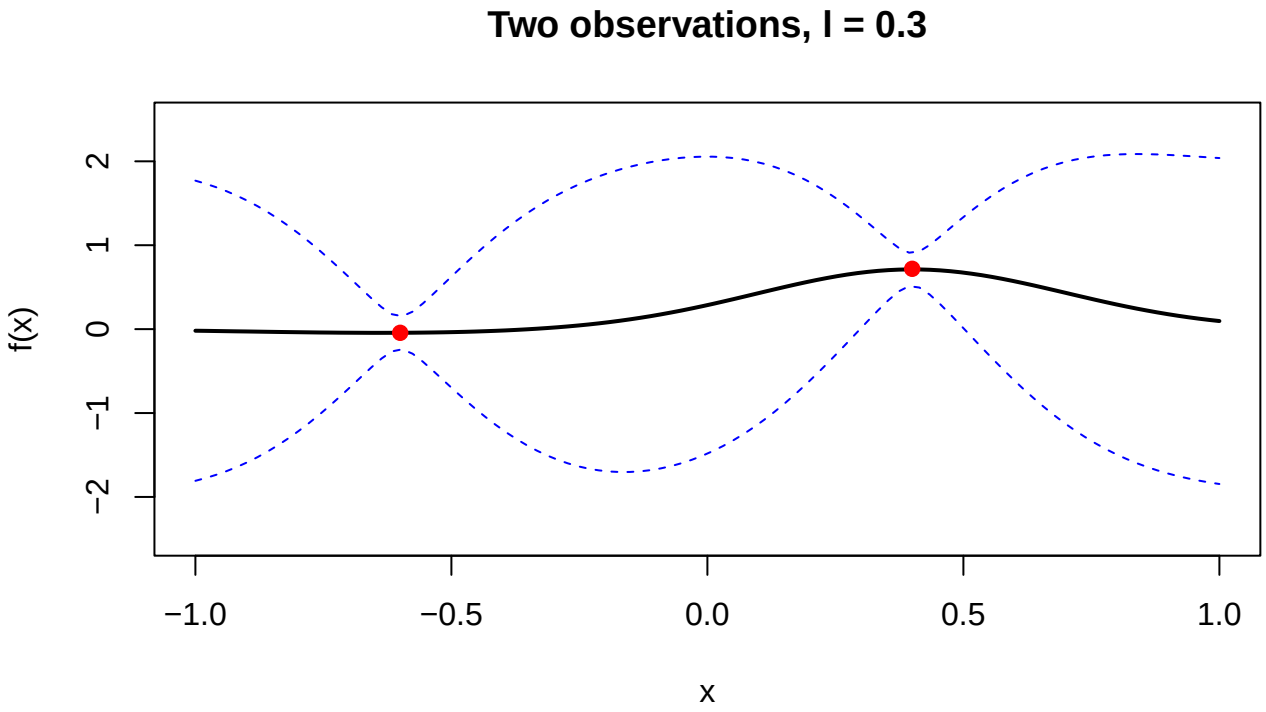
As a check, with one observation at $(0.4, 0.719)$ the posterior mean at $x = 0.4$ is $0.719/1.01 \approx 0.712$ and the variance is $1 - 1/1.01 \approx 0.0099$, which matches what the function returns.

2.1 (2) One observation



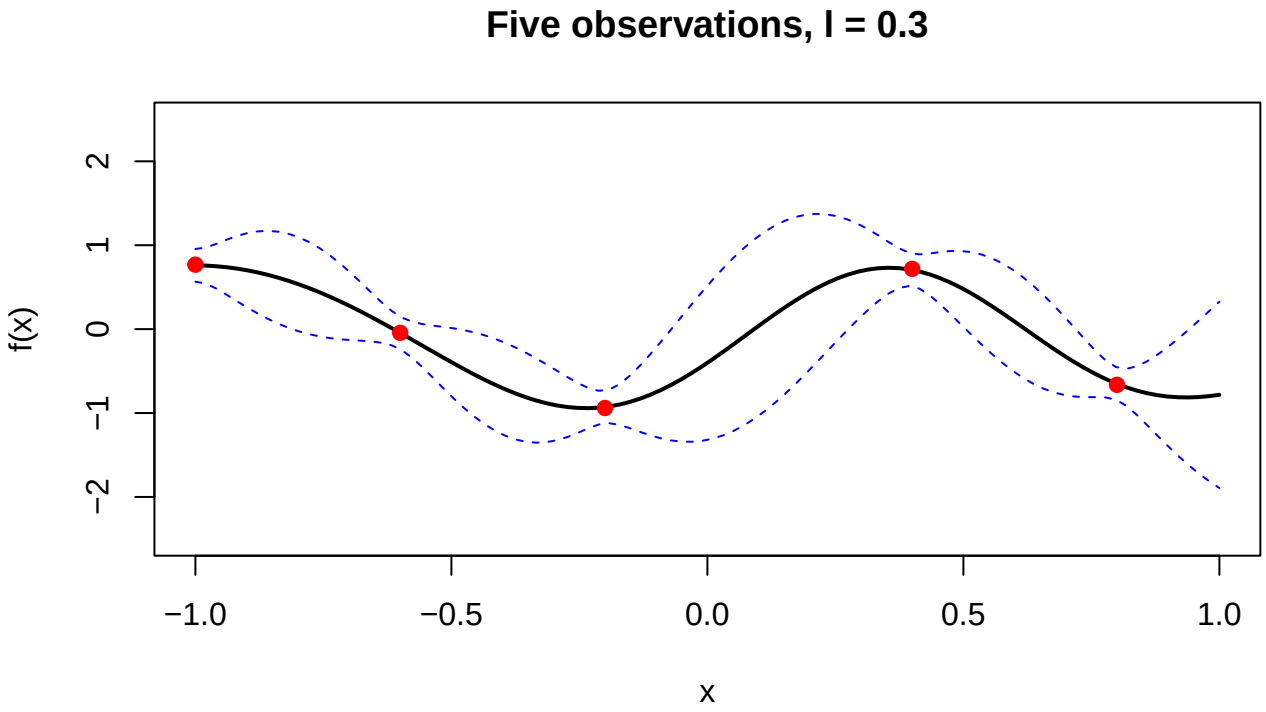
The mean goes close to the observation and the band is narrow there. Further away the mean goes back to 0 and the band widens to the prior, ± 1.96, since the point has no influence more than about 3ℓ away.

2.1 (3) Two observations



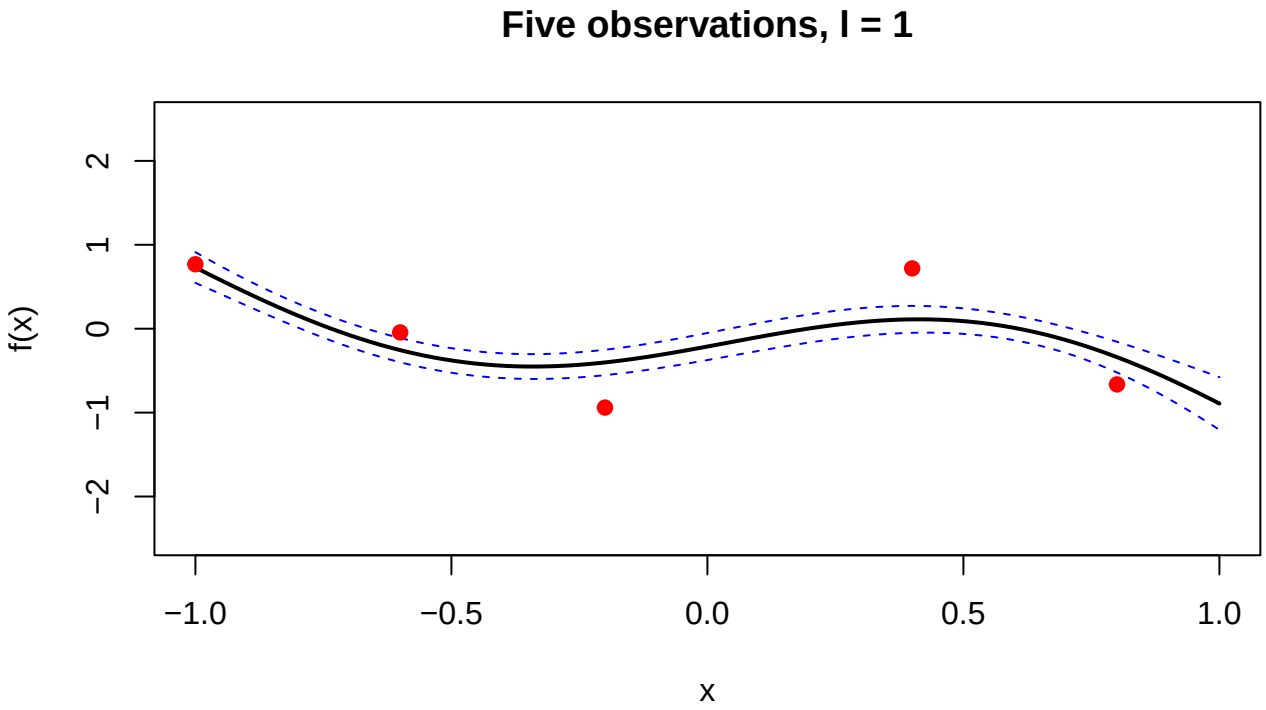
The new point only changes the posterior near itself. The two points are 1 apart (more than 3ℓ), so they are almost uncorrelated and the curve around $x = 0.4$ is the same as before.

2.1 (4) Five observations



With five points the mean follows the data closely and the band is narrow wherever there is data nearby. It only gets wider in the gaps between points.

2.1 (5) Five observations, $\ell = 1$



With $\ell = 1$ the prior assumes a much smoother function. Each point now affects a large part of the interval, so the bands are narrower, but the mean is too stiff to follow the data, which goes up and down a lot over short distances. It misses most of the points. For this data $\ell = 0.3$ gives a much better fit.

2.2 GP Regression with kernlab

2.2 (1) Own SE kernel

The kernel is written in the kernlab format: an outer function that takes σ_f and ℓ and returns the kernel function, labeled with class `"kernel"`. With $\sigma_f = 1$ and $\ell = 1$, the kernel at $x = 1$, $x' = 2$ is:

```
## [1] 0.6065307
```

and the covariance matrix $K(X, X_*)$ for $X = (1, 3, 4)^T$ and $X_* = (2, 3, 4)^T$ is:

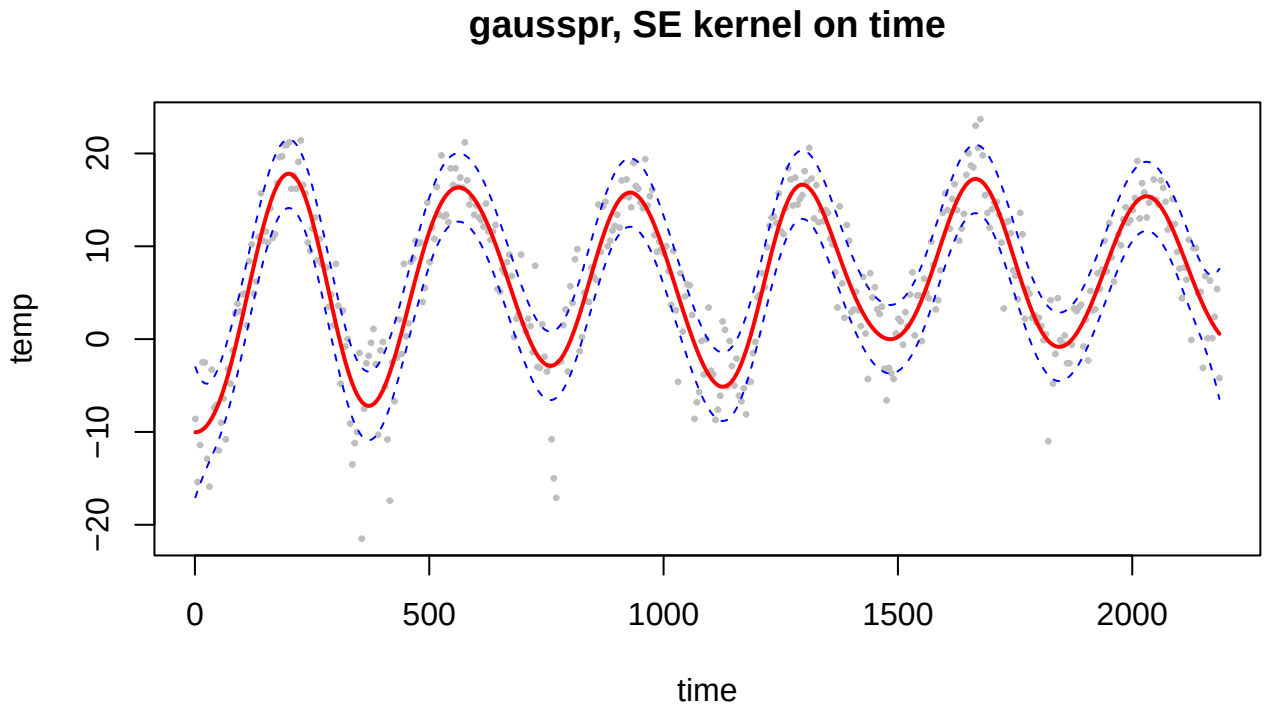
```
## An object of class "kernelMatrix"
##        [,1]   [,2]   [,3]
## [1,] 0.6065 0.1353 0.0111
## [2,] 0.6065 1.0000 0.6065
## [3,] 0.1353 0.6065 1.0000
```

The entries are 1 where the inputs are equal and get smaller with distance.

2.2 (2) Model with time

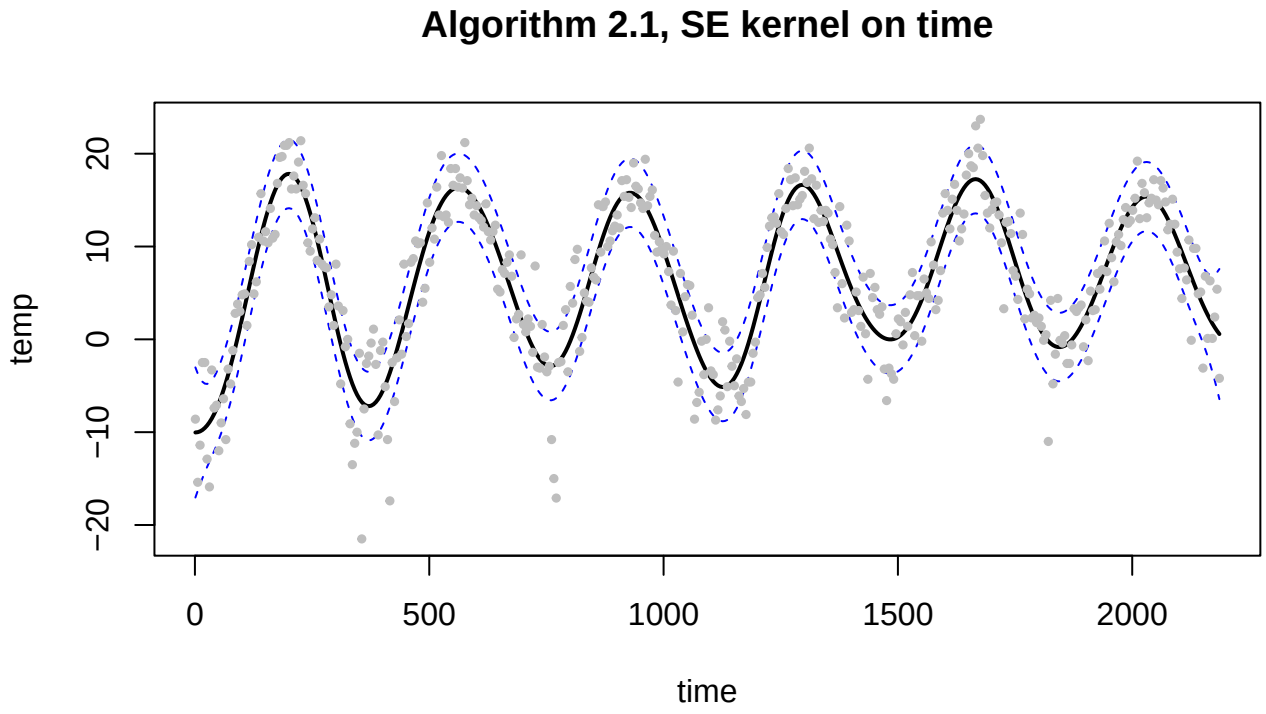
The noise variance σ_n^2 is the residual variance from a quadratic regression of temperature on time:

```
## [1] 66.85169
```



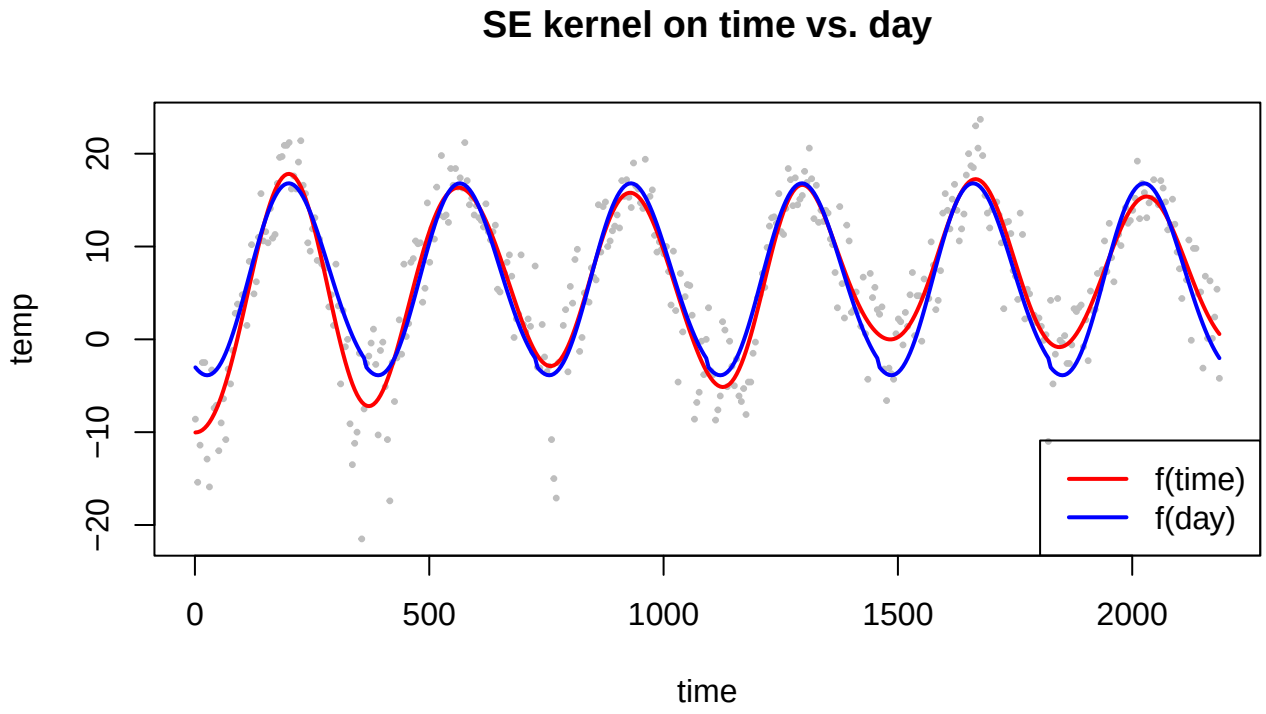
The variance is quite large (sd about 8 degrees) because a quadratic over six years cannot follow the seasons, so the residuals include the seasonal swing as well as the day-to-day weather. The posterior mean follows the yearly cycle smoothly. With $\ell = 100$ days it picks up the seasons but not the short-term variation.

2.2 (3) Same model with Algorithm 2.1



The posterior mean and the bands are the same as from `gausspr`, which confirms that `posteriorGP` is correct. Many points are outside the bands, but that is expected. The bands are for f, the smooth underlying temperature, and not for the individual measurements, which scatter around f with the noise. A band for new observations would be mean $\pm\, 1.96\sqrt{\text{var} + \sigma_n^2}$.

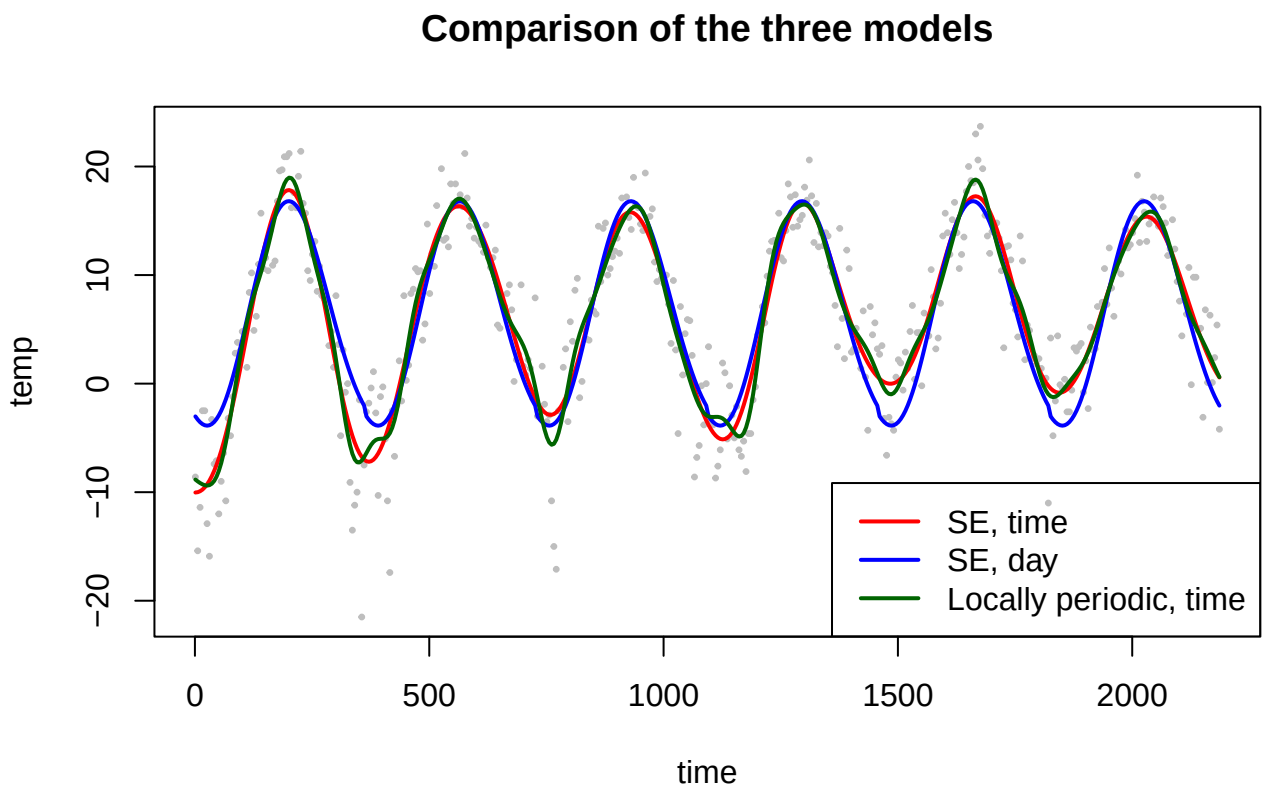
2.2 (4) Model with day



The day model gives the same curve every year. All six measurements of the same date get the same input, so the model pools them into one average seasonal profile (winters around -4, summers around 17).

Pros and cons. The day model uses all six years to estimate each day of the year, so the seasonal pattern is stable and it can be used to predict future years. But it cannot capture differences between years, like the cold winters at the start and in 2010/11 or the mild winter around time 1500. The year-to-year variation is just treated as noise. It also treats 31 December and 1 January as far apart (distance 364), even though they are neighbours. The time model follows each year's actual data, but it does not know that the pattern repeats. Each year is only learned from nearby data, and outside the data it would go back to the prior mean 0.

2.2 (5) Locally periodic kernel



The locally periodic model follows the data more closely than the other two. It has higher summer peaks, lower winter lows, and some small bumps that the SE models smooth out (around time 400, 750 and 1100).

This is because the kernel is less smooth for nearby points. For small distances the periodic part with $\ell_1 = 1$ behaves like an SE kernel with length scale $d/(2\pi) \approx 58$ days, and multiplied with the SE part ($\ell_2 = 100$) the combined length scale is about 50 days, half of the 100 used in the other models. The bumps show up mostly in winter because the winter data is much more spread out, with cold snaps down to -15 to -20, and the more flexible model bends toward them. Summer data is more tightly clustered, so the curve stays smooth there.

The yearly part does not do much with these hyperparameters. One year apart, the SE factor with $\ell_2 = 100$ is almost zero ($k(1, 366) \approx 0.5$ compared to 400 at distance 0), so the years barely share information. The result is mainly a more flexible version of the time model, which still captures differences between years, unlike the day model. A larger ℓ_2 would let the periodic part carry information between years and combine the strengths of both earlier models.

2.3 GP Classification with kernlab

2.3 (1) Two covariates

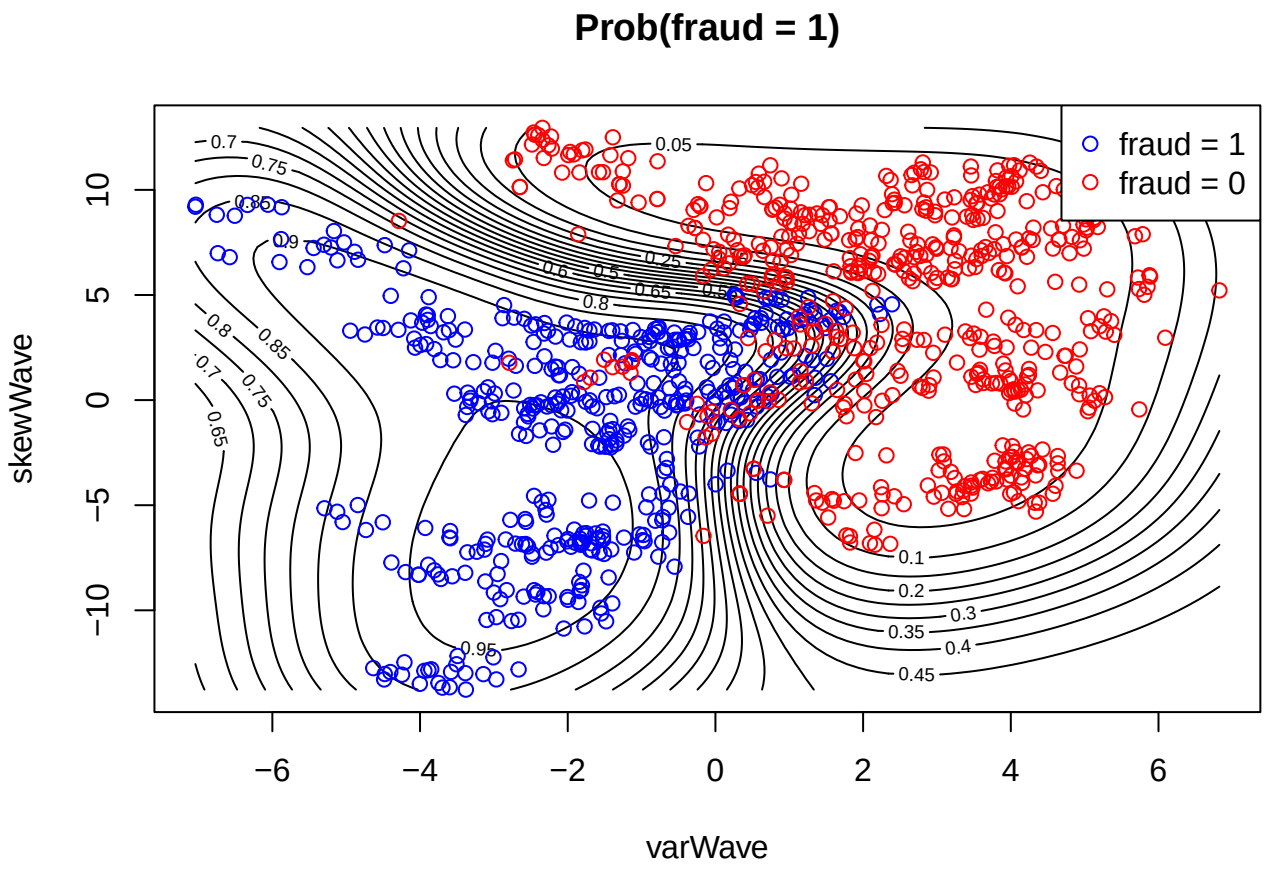
The model uses the default kernel (RBF, with σ chosen automatically by `sigest`) and only `varWave` and `skewWave`.

```
## Using automatic sigma estimation (sigest) for RBF or laplace kernel
```

```
##       predicted
## actual   0   1
##      0 503  41
##      1  18 438
```

```
## [1] 0.941
```

The training accuracy is 0.941.



The probability of fraud is high (about 0.9) around the fraud points on the left and low (about 0.05) around the genuine notes on the right. The decision boundary is near `varWave` = 0, where the two classes overlap, which is where the errors come from. Far away from the data the probabilities move back toward 0.5, since the GP goes back to its prior where there is no information.

2.3 (2) Test set

```
##       predicted
## actual   0   1
```

```
##      0 199  19
##      1   9 145
```

```
## [1] 0.9247312
```

The test accuracy is 0.925, slightly lower than on the training data, as expected. The difference is small, so the model is not overfitting much.

2.3 (3) All four covariates

```
## Using automatic sigma estimation (sigest) for RBF or laplace kernel
```

```
##       predicted
## actual   0   1
##      0 216   2
##      1   0 154
```

```
## [1] 0.9946237
```

With all four covariates the test accuracy is 0.995, compared to 0.925 with two. Only 2 of the 372 test notes are misclassified. `kurtWave` and `entropyWave` contain information that separates the classes much better, especially in the region where the two classes overlap when only `varWave` and `skewWave` are used.

Appendix: R Code

```
knitr::opts_chunk$set(echo = FALSE, message = FALSE, warning = FALSE,
                      fig.width = 7, fig.height = 4.2, fig.align = "center")
library(kernlab)
# Squared exponential kernel (from GaussianProcesses.R on the course page)
SquaredExpKernel <- function(x1, x2, sigmaF = 1, l = 3) {
  n1 <- length(x1)
  n2 <- length(x2)
  K <- matrix(NA, n1, n2)
  for (i in 1:n2) {
    K[, i] <- sigmaF^2 * exp(-0.5 * ((x1 - x2[i]) / l)^2)
  }
  return(K)
}

# Algorithm 2.1 (Rasmussen and Williams)
posteriorGP <- function(X, y, XStar, sigmaNoise, k) {
  K <- k(X, X)
  L <- t(chol(K + sigmaNoise^2 * diag(nrow(K))))
  alpha <- solve(t(L), solve(L, y))
  k_star <- k(X, XStar)
  f_star_mean <- t(k_star) %*% alpha
  v <- solve(L, k_star)
  f_star_var <- diag(k(XStar, XStar)) - diag(t(v) %*% v)
  return(list(mean = f_star_mean, var = f_star_var))
}

# Plot posterior mean with 95% pointwise bands
plotGP <- function(X, y, XStar, res, main = "", ylim = c(-2.5, 2.5),
                   xlab = "x", ylab = "f(x)", ptcol = "red", ptcex = 1) {
  sd <- sqrt(res$var)
  plot(XStar, res$mean, type = "l", lwd = 2, ylim = ylim,
       xlab = xlab, ylab = ylab, main = main)
  lines(XStar, res$mean + 1.96 * sd, col = "blue", lty = 2)
  lines(XStar, res$mean - 1.96 * sd, col = "blue", lty = 2)
  points(X, y, pch = 19, col = ptcol, cex = ptcex)
}

XStar <- seq(-1, 1, length.out = 100)
k03 <- function(x1, x2) SquaredExpKernel(x1, x2, sigmaF = 1, l = 0.3)
k1  <- function(x1, x2) SquaredExpKernel(x1, x2, sigmaF = 1, l = 1)
X <- 0.4; y <- 0.719
res <- posteriorGP(X, y, XStar, sigmaNoise = 0.1, k = k03)
plotGP(X, y, XStar, res, main = "One observation, l = 0.3")
X <- c(0.4, -0.6); y <- c(0.719, -0.044)
res <- posteriorGP(X, y, XStar, sigmaNoise = 0.1, k = k03)
plotGP(X, y, XStar, res, main = "Two observations, l = 0.3")
X <- c(-1.0, -0.6, -0.2, 0.4, 0.8)
y <- c(0.768, -0.044, -0.940, 0.719, -0.664)
res <- posteriorGP(X, y, XStar, sigmaNoise = 0.1, k = k03)
plotGP(X, y, XStar, res, main = "Five observations, l = 0.3")
res <- posteriorGP(X, y, XStar, sigmaNoise = 0.1, k = k1)
```

```r
plotGP(X, y, XStar, res, main = "Five observations, l = 1")
data <- read.csv("https://github.com/STIMALiU/AdvMLCourse/raw/master/GaussianProcess/Code/TempTullinge.
                 header = TRUE, sep = ";")
time <- 1:2190
day <- rep(1:365, 6)
idx <- seq(1, 2190, by = 5)
time <- time[idx]; day <- day[idx]; temp <- data$temp[idx]
SEkernelKL <- function(sigmaf = 1, ell = 1) {
  rval <- function(x, y = NULL) {
    r <- sqrt(sum((x - y)^2))
    return(sigmaf^2 * exp(-(r^2 / (2 * ell^2))))
  }
  class(rval) <- "kernel"
  return(rval)
}

kern <- SEkernelKL(sigmaf = 1, ell = 1)
kern(1, 2)
round(as.matrix(kernelMatrix(kern, c(1, 3, 4), c(2, 3, 4))), 4)
fit <- lm(temp ~ time + I(time^2))
sigmaNoise <- var(fit$residuals)   # variance, not sd
sigmaNoise

kern_20_100 <- SEkernelKL(sigmaf = 20, ell = 100)
GPfit <- gausspr(time, temp, kernel = kern_20_100, var = sigmaNoise,
                 variance.model = TRUE, scaled = FALSE)
meanPred <- predict(GPfit, time)
sdPred <- predict(GPfit, time, type = "sdeviation")

plot(time, temp, pch = 20, cex = 0.5, col = "grey", main = "gausspr, SE kernel on time")
lines(time, meanPred, col = "red", lwd = 2)
lines(time, meanPred + 1.96 * sdPred, col = "blue", lty = 2)
lines(time, meanPred - 1.96 * sdPred, col = "blue", lty = 2)
k_20_100 <- function(x1, x2) SquaredExpKernel(x1, x2, sigmaF = 20, l = 100)
res <- posteriorGP(time, temp, time, sigmaNoise = sqrt(sigmaNoise), k = k_20_100)
plotGP(time, temp, time, res, main = "Algorithm 2.1, SE kernel on time",
       ylim = range(temp), xlab = "time", ylab = "temp", ptcol = "grey", ptcex = 0.5)
GPfit_day <- gausspr(day, temp, kernel = kern_20_100, var = sigmaNoise,
                     variance.model = TRUE, scaled = FALSE)
meanPred_day <- predict(GPfit_day, day)

plot(time, temp, pch = 20, cex = 0.5, col = "grey", main = "SE kernel on time vs. day")
lines(time, meanPred, col = "red", lwd = 2)
lines(time, meanPred_day, col = "blue", lwd = 2)
legend("bottomright", legend = c("f(time)", "f(day)"), col = c("red", "blue"), lwd = 2)
SELPkernelKL <- function(sigmaf = 1, ell1 = 1, ell2 = 1, d = 365) {
  rval <- function(x, y = NULL) {
    r <- abs(x - y)
    return(sigmaf^2 * exp(-2 * sin(pi * r / d)^2 / ell1^2) * exp(-0.5 * r^2 / ell2^2))
  }
  class(rval) <- "kernel"
  return(rval)
}
```

```r
kern_lp <- SELPkernelKL(sigmaf = 20, ell1 = 1, ell2 = 100, d = 365)
GPfit_lp <- gausspr(time, temp, kernel = kern_lp, var = sigmaNoise,
                    variance.model = TRUE, scaled = FALSE)
meanPred_lp <- predict(GPfit_lp, time)

plot(time, temp, pch = 20, cex = 0.5, col = "grey", main = "Comparison of the three models")
lines(time, meanPred, col = "red", lwd = 2)
lines(time, meanPred_day, col = "blue", lwd = 2)
lines(time, meanPred_lp, col = "darkgreen", lwd = 2)
legend("bottomright", legend = c("SE, time", "SE, day", "Locally periodic, time"),
       col = c("red", "blue", "darkgreen"), lwd = 2)
data <- read.csv("https://github.com/STIMALiU/AdvMLCourse/raw/master/GaussianProcess/Code/banknoteFraud
                 header = FALSE, sep = ",")
names(data) <- c("varWave", "skewWave", "kurtWave", "entropyWave", "fraud")
data[, 5] <- as.factor(data[, 5])

set.seed(111)
SelectTraining <- sample(1:dim(data)[1], size = 1000, replace = FALSE)
train <- data[SelectTraining, ]
test <- data[-SelectTraining, ]
GPfitFraud <- gausspr(fraud ~ varWave + skewWave, data = train)

pred_train <- predict(GPfitFraud, train[, c("varWave", "skewWave")])
cm_train <- table(actual = train$fraud, predicted = pred_train)
accuracy_train <- sum(diag(cm_train)) / sum(cm_train)
cm_train
accuracy_train
x1 <- seq(min(train$varWave), max(train$varWave), length = 100)
x2 <- seq(min(train$skewWave), max(train$skewWave), length = 100)
gridPoints <- expand.grid(varWave = x1, skewWave = x2)
probPreds <- predict(GPfitFraud, gridPoints, type = "probabilities")

contour(x1, x2, matrix(probPreds[, 2], 100, 100), 20,
        xlab = "varWave", ylab = "skewWave", main = "Prob(fraud = 1)")
points(train$varWave[train$fraud == 1], train$skewWave[train$fraud == 1], col = "blue")
points(train$varWave[train$fraud == 0], train$skewWave[train$fraud == 0], col = "red")
legend("topright", legend = c("fraud = 1", "fraud = 0"),
       col = c("blue", "red"), pch = 1, bg = "white")
pred_test <- predict(GPfitFraud, test[, c("varWave", "skewWave")])
cm_test <- table(actual = test$fraud, predicted = pred_test)
accuracy_test <- sum(diag(cm_test)) / sum(cm_test)
cm_test
accuracy_test
GPfitFraud_4 <- gausspr(fraud ~ ., data = train)

pred_test_4 <- predict(GPfitFraud_4, test[, c("varWave", "skewWave", "kurtWave", "entropyWave")])
cm_test_4 <- table(actual = test$fraud, predicted = pred_test_4)
accuracy_test_4 <- sum(diag(cm_test_4)) / sum(cm_test_4)
cm_test_4
accuracy_test_4
```